Programming in Java Evaluated Course Project

HOSPITAL MANAGEMENT SYSTEM

A Console-Based Java Application

Built using Core Java | OOP Principles | JDK 11+

Project Title	Hospital Management System (HMS)
Language	Java
Student Name	Aditya Jayant
Registration No	24BAI10579
Branch	B.Tech – Artificial Intelligence & Machine Learning
Submission Date	31st March 2026
Slot	A21+A22
Faculty	Nirmala.E

1. Introduction

The Hospital Management System (HMS) is a console-based Java application built to automate the daily operations of a hospital. It manages patients, doctors, appointments, and billing records through a clean, menu-driven interface. The project was developed using only core Java with no external libraries or frameworks, making it easy to compile and run on any system with JDK 11 or higher.

Hospitals and clinics deal with large amounts of data every day. Tracking patient admissions, doctor assignments, scheduled appointments, and pending bills manually is slow and error-prone. This project simulates that environment in a structured way, showing how Object-Oriented Programming can be used to model real-world entities and their relationships.

1.1 Objectives

- Build a fully functional Hospital Management System using Java OOP principles
- Implement five modules: Patient, Doctor, Appointment, Billing, and Dashboard
- Demonstrate encapsulation, abstraction, and modularity in a practical project
- Use Java Collections (HashMap, ArrayList) and Streams for all data operations
- Provide a colour-coded, user-friendly console interface using ANSI escape codes

1.2 Scope

The system covers the complete workflow of a small hospital or clinic from patient admission to discharge and billing. All data is stored in memory using Java Collections during runtime. The application can be extended in future with a MySQL database via JDBC, a desktop GUI using Swing or JavaFX, or a REST API using Spring Boot.

2. System Requirements

2.1 Hardware Requirements

- Processor: Any modern CPU (Intel / AMD / ARM)
- RAM: Minimum 512 MB (1 GB recommended)
- Storage: At least 50 MB free disk space

2.2 Software Requirements

- Operating System: Windows 10/11, macOS, or any Linux distribution
- Java Development Kit (JDK): Version 11 or higher
- Text Editor or IDE: VS Code, IntelliJ IDEA, Eclipse, or Notepad++
- Terminal: Command Prompt on Windows, or Bash on Linux/macOS

2.3 Steps to Compile and Run

On Windows, wildcards do not work in Command Prompt, so all files must be listed explicitly. Follow these steps from the project root folder:

```
Step 1:  cd C:\Users\YourName\HospitalManagementSystem
Step 2:  mkdir out
Step 3:  javac -d out src\model\Patient.java src\model\Doctor.java
src\model\Appointment.java src\model\Bill.java src\service\PatientService.java
src\service\DoctorService.java src\service\AppointmentService.java
src\service\BillingService.java src\util\ConsoleUI.java src\util\DataSeeder.java
src\HospitalManagementSystem.java
Step 4:  cd out
Step 5:  java HospitalManagementSystem
```

On Linux or macOS, the provided run.sh script handles compilation and execution in one command:

```
chmod +x run.sh
./run.sh
```

3. Project Structure

The project is divided into three packages plus a main class. Each package has a single responsibility. This design follows the principle of Separation of Concerns and makes the code easy to understand, test, and extend.

3.1 Folder Structure

```
HospitalManagementSystem/
  src/
    HospitalManagementSystem.java    <-- Main class (entry point)
    model/
      Patient.java
      Doctor.java
      Appointment.java
      Bill.java
    service/
      PatientService.java
      DoctorService.java
      AppointmentService.java
      BillingService.java
    util/
      ConsoleUI.java
      DataSeeder.java
  out/              <-- Compiled .class files go here
  run.sh            <-- Build and run script (Linux/macOS)
```

3.2 Class Summary

The following table describes the purpose of every Java file in the project:

Class / File	Package	Description
Patient.java	model	Stores patient info – ID, name, age, blood group, disease, admission date.
Doctor.java	model	Stores doctor info – specialization, experience, list of assigned patients.
Appointment.java	model	Links patient + doctor with date/time, reason, and a Status enum.
Bill.java	model	Itemised charges in a LinkedHashMap, tracks payments and PaymentStatus.
PatientService.java	service	Handles add, search, discharge, and delete for patients.
DoctorService.java	service	Handles add, search, and patient assignment for doctors.
AppointmentService.java	service	Schedule, complete, cancel appointments; filter by patient or doctor.
BillingService.java	service	Auto bill generation with standard rates, payment processing.

ConsoleUI.java	util	ANSI colour helpers: printSuccess, printError, readInt, readString.
DataSeeder.java	util	Pre-loads 5 doctors, 5 patients, 4 appointments, 3 bills on startup.
HospitalManagementSystem.java	–	Main entry point. Menu-driven UI that calls service methods.

4. OOP Concepts Applied

This project was designed to demonstrate Object-Oriented Programming in a practical, real-world context. The table below shows which OOP concept is used, where it appears, and what purpose it serves in the system.

OOP Concept	Where Used	Purpose
Encapsulation	Model classes	All fields are private; accessed only via getters/setters.
Abstraction	Service layer	Callers use addPatient() without knowing the HashMap internals.
Modularity	Package structure	model / service / util – each class has one clear responsibility.
Enum Usage	Status fields in model	Appointment.Status and Bill.PaymentStatus drive state changes.
Java Streams	Service filter methods	filter() + collect() replaces verbose for-loops throughout.

4.1 Encapsulation

All fields in every model class are declared private. They can only be accessed or modified through public getter and setter methods. This protects the internal state of objects from being changed accidentally from outside the class.

```
public class Patient {
    private int    patientId;
    private String name;
    private String disease;
    private LocalDate admissionDate;
    private boolean admitted;

    public String getName()            { return name; }
    public void   setName(String name) { this.name = name; }

    // Calculates days in hospital using java.time.Period
    public int getDaysAdmitted() {
        return Period.between(admissionDate, LocalDate.now()).getDays();
    }
}
```

4.2 Abstraction

The service layer hides all internal data structures from the UI. The main class does not know that patients are stored in a HashMap. It simply calls addPatient() and gets a result.

```
public Patient addPatient(String name, int age, String gender,
                          String bloodGroup, String phone,
                          String address, String disease) {
    Patient p = new Patient(name, age, gender, bloodGroup,
```

```
                                phone, address, disease);
    patients.put(p.getPatientId(), p);   // internal detail hidden
    return p;
}
```

4.3 Java Streams

Java Streams are used throughout the service layer to filter collections. This avoids writing long for-loops and makes the code much more concise and readable.

```
// Search patients by disease using Java Streams
public List<Patient> searchByDisease(String disease) {
    String lower = disease.toLowerCase();
    return patients.values().stream()
        .filter(p -> p.getDisease().toLowerCase().contains(lower))
        .collect(Collectors.toList());
}
```

4.4 Enums for State Management

Enums are used to represent the state of appointments and bills. This is cleaner and safer than using String comparisons or integer codes.

```
// In Appointment.java
public enum Status { SCHEDULED, COMPLETED, CANCELLED }

// In Bill.java
public enum PaymentStatus { PENDING, PAID, PARTIALLY_PAID }

// makePayment() automatically updates the status
public void makePayment(double amount) {
    paidAmount += amount;
    if      (paidAmount >= totalAmount) paymentStatus = PaymentStatus.PAID;
    else if (paidAmount > 0)            paymentStatus = PaymentStatus.PARTIALLY_PAID;
}
```

5. Modules and Features

The system is divided into five modules. Each module is accessible from the main menu and is backed by its own service class. The table below gives an overview of all five modules.

No.	Module	Functionality	Java File
1	**Patient Management**	Add, view, search, discharge, delete patients	PatientService.java
2	**Doctor Management**	Add doctors, assign/remove patients from doctors	DoctorService.java
3	**Appointment Scheduling**	Book, complete, and cancel appointments	AppointmentService.java
4	**Billing & Payments**	Auto bill generation, payments, revenue summary	BillingService.java
5	**Dashboard**	Live hospital stats and financial overview	HospitalManagementSystem.java

5.1 Patient Management

- Add a new patient – name, age, gender, blood group, phone, address, disease
- View all patients or only currently admitted patients in a formatted table
- Search patients by name or disease (case-insensitive partial match)
- View complete patient details including number of days admitted
- Discharge a patient – marks them as discharged but keeps the record for billing
- Delete a patient record permanently

```
╔═══════════════════════════════════════════════════════════════╗
║          HOSPITAL MANAGEMENT SYSTEM  v1.0                     ╖
║          Powered by Java  |  City General Hospital            ╖
╚═══════════════════════════════════════════════════════════════╝

? Sample data has been loaded. Use the menu to get started.

Press ENTER to continue...
======================================================================
                              MAIN MENU
======================================================================
  [1] Patient Management
  [2] Doctor Management
  [3] Appointment Management
  [4] Billing & Payments
  [5] Dashboard / Reports
  [0] Exit
----------------------------------------------------------------------
Enter choice: 1
======================================================================
                          PATIENT MANAGEMENT
======================================================================
  [1] Add New Patient
  [2] View All Patients
  [3] View Admitted Patients
  [4] Search Patient by Name
  [5] Search Patient by Disease
  [6] View Patient Details
  [7] Discharge Patient
  [8] Delete Patient Record
  [0] Back to Main Menu
----------------------------------------------------------------------
Enter choice: 3
```

```
----------------------------------------------------------------------
                          PATIENT MANAGEMENT
======================================================================
  [1] Add New Patient
  [2] View All Patients
  [3] View Admitted Patients
  [4] Search Patient by Name
  [5] Search Patient by Disease
  [6] View Patient Details
  [7] Discharge Patient
  [8] Delete Patient Record
  [0] Back to Main Menu
----------------------------------------------------------------------
Enter choice: 3

--- ADMITTED PATIENTS ---

+---------+----------------------+------+---------+------------+-----------------+--------------------------+
| Pat.ID  | Name                 | Age  | Gender  | Blood Grp  | Phone           | Disease                  |
+---------+----------------------+------+---------+------------+-----------------+--------------------------+
| 1001    | Ramesh Kumar         | 45   | Male    | A+         | 9812345678      | Hypertension             |
| 1002    | Sita Devi            | 32   | Female  | B+         | 9812345679      | Migraine                 |
| 1003    | Arjun Patel          | 28   | Male    | O+         | 9812345680      | Fracture - Left Leg      |
| 1004    | Meena Joshi          | 7    | Female  | AB+        | 9812345681      | Fever & Cold             |
| 1005    | Vikram Singh         | 60   | Male    | A-         | 9812345682      | Chest Pain               |
+---------+----------------------+------+---------+------------+-----------------+--------------------------+
? Total: 5 patient(s)
Press ENTER to continue...7
======================================================================
                          PATIENT MANAGEMENT
======================================================================
  [1] Add New Patient
  [2] View All Patients
  [3] View Admitted Patients
  [4] Search Patient by Name
  [5] Search Patient by Disease
  [6] View Patient Details
  [7] Discharge Patient
  [8] Delete Patient Record
  [0] Back to Main Menu
----------------------------------------------------------------------
Enter choice: 7
Enter Patient ID to discharge: 1001
? Patient discharged successfully.

Press ENTER to continue...8
======================================================================
                          PATIENT MANAGEMENT
======================================================================
  [1] Add New Patient
  [2] View All Patients
  [3] View Admitted Patients
  [4] Search Patient by Name
  [5] Search Patient by Disease
  [6] View Patient Details
  [7] Discharge Patient
  [8] Delete Patient Record
  [0] Back to Main Menu
----------------------------------------------------------------------
Enter choice: 1

--- ADD NEW PATIENT ---
Full Name        : aditya jayant
Age              : 20
Gender (M/F/O)   : m
Blood Group      : a+
Phone Number     : 9315980125
Address          : vit bhopal
Disease/Condition: allergy
? Patient admitted! ID: 1006
```

```
Press ENTER to continue...4
======================================================================
                          PATIENT MANAGEMENT
======================================================================
  [1] Add New Patient
  [2] View All Patients
  [3] View Admitted Patients
  [4] Search Patient by Name
  [5] Search Patient by Disease
  [6] View Patient Details
  [7] Discharge Patient
  [8] Delete Patient Record
  [0] Back to Main Menu
----------------------------------------------------------------------
Enter choice: 4
Enter name to search: aditya jayant

--- SEARCH RESULTS ---

+---------+----------------------+------+---------+------------+-----------------+---------------------------+
| Pat.ID  | Name                 | Age  | Gender  | Blood Grp  | Phone           | Disease                   |
+---------+----------------------+------+---------+------------+-----------------+---------------------------+
| 1006    | aditya jayant        | 20   | m       | a+         | 9315980125      | allergy                   |
+---------+----------------------+------+---------+------------+-----------------+---------------------------+

Press ENTER to continue...
======================================================================
                          PATIENT MANAGEMENT
======================================================================
  [1] Add New Patient
  [2] View All Patients
  [3] View Admitted Patients
  [4] Search Patient by Name
  [5] Search Patient by Disease
  [6] View Patient Details
  [7] Discharge Patient
  [8] Delete Patient Record
  [0] Back to Main Menu
----------------------------------------------------------------------
Enter choice: 5
Enter disease to search: allergy

--- SEARCH RESULTS ---

+---------+----------------------+------+---------+------------+-----------------+---------------------------+
| Pat.ID  | Name                 | Age  | Gender  | Blood Grp  | Phone           | Disease                   |
+---------+----------------------+------+---------+------------+-----------------+---------------------------+
| 1006    | aditya jayant        | 20   | m       | a+         | 9315980125      | allergy                   |
+---------+----------------------+------+---------+------------+-----------------+---------------------------+
? Total: 1 patient(s)
```

5.2 Doctor Management

- Add a doctor with name, specialization, experience (years), phone, email, availability
- View all doctors showing their current patient count
- Search doctors by specialization (e.g., Cardiology, Neurology)
- View a doctor's full profile including list of assigned patient IDs
- Assign or remove a patient from a doctor's list

```
==========================================================================
                          DOCTOR MANAGEMENT
==========================================================================
  [1] Add New Doctor
  [2] View All Doctors
  [3] Search by Specialization
  [4] View Doctor Details & Patients
  [5] Assign Patient to Doctor
  [6] Remove Patient from Doctor
  [0] Back to Main Menu
--------------------------------------------------------------------------
Enter choice: 1

--- ADD NEW DOCTOR ---
Full Name          : aditya jayant
Specialization     : ent
Phone              : 999999999
Email              : ajdsd@gmail.com
Experience (yrs)   : 10
Availability       : yes
? Doctor added! ID: 206

Press ENTER to continue...
```

-

```
==================================================================
                        DOCTOR MANAGEMENT
==================================================================
  [1] Add New Doctor
  [2] View All Doctors
  [3] Search by Specialization
  [4] View Doctor Details & Patients
  [5] Assign Patient to Doctor
  [6] Remove Patient from Doctor
  [0] Back to Main Menu
------------------------------------------------------------------
Enter choice: 2

--- DOCTOR LIST ---

+--------+---------------------+---------------------+----------+----------------+----------+
| Doc.ID | Name                | Specialization      | Exp(Yrs) | Phone          | Patients |
+--------+---------------------+---------------------+----------+----------------+----------+
| 201    | Dr. Anil Sharma     | Cardiology          | 15   yrs | 9876543210     | 2        |
| 202    | Dr. Priya Verma     | Neurology           | 10   yrs | 9876543211     | 1        |
| 203    | Dr. Rahul Gupta     | Orthopedics         | 12   yrs | 9876543212     | 1        |
| 204    | Dr. Sunita Patel    | Pediatrics          | 8    yrs | 9876543213     | 1        |
| 205    | Dr. Manoj Singh     | General Surgery     | 20   yrs | 9876543214     | 0        |
| 206    | aditya jayant       | ent                 | 10   yrs | 999999999      | 0        |
+--------+---------------------+---------------------+----------+----------------+----------+

Press ENTER to continue...
```

-

```
Press ENTER to continue...
==========================================================================
                          DOCTOR MANAGEMENT
==========================================================================
  [1] Add New Doctor
  [2] View All Doctors
  [3] Search by Specialization
  [4] View Doctor Details & Patients
  [5] Assign Patient to Doctor
  [6] Remove Patient from Doctor
  [0] Back to Main Menu
--------------------------------------------------------------------------
Enter choice: 6
Enter Doctor ID  : 206
Enter Patient ID : 1003
? Patient removed from doctor.
```

-

```
======================================================================
                          DOCTOR MANAGEMENT
======================================================================
  [1] Add New Doctor
  [2] View All Doctors
  [3] Search by Specialization
  [4] View Doctor Details & Patients
  [5] Assign Patient to Doctor
  [6] Remove Patient from Doctor
  [0] Back to Main Menu
----------------------------------------------------------------------
Enter choice: 5
Enter Doctor ID  : 206
Enter Patient ID : 1002
? Patient assigned to doctor.

Press ENTER to continue...
```

-

5.3 Appointment Scheduling

- Schedule an appointment between any patient and any doctor with date, time, and reason
- View all appointments or filter by patient ID or doctor ID
- View only upcoming (SCHEDULED) appointments
- Mark an appointment as completed and add clinical notes from the doctor
- Cancel any scheduled appointment

```
======================================================================
                       APPOINTMENT MANAGEMENT
======================================================================
  [1] Schedule New Appointment
  [2] View All Appointments
  [3] View Scheduled (Upcoming)
  [4] View by Patient ID
  [5] View by Doctor ID
  [6] Complete Appointment
  [7] Cancel Appointment
  [0] Back to Main Menu
----------------------------------------------------------------------
Enter choice: 1

--- SCHEDULE APPOINTMENT ---
Patient ID : 1002
Doctor ID  : 206
Date & Time (dd-MM-yyyy HH:mm): 31-03-2026 12:30
Reason: allergy
? Appointment scheduled! ID: 5005

Press ENTER to continue...
```

5.4 Billing and Payments

Bills are auto-generated using the following standard charge rates:

- Consultation Fee: Rs. 500
- Bed Charges: Rs. 1,500 per day (based on days admitted)
- Medicines: Rs. 2,000
- Laboratory Tests: Rs. 800

- Surgery Fee (if applicable): Rs. 25,000

Payments can be made in full or in instalments. The bill status updates automatically to PENDING, PARTIALLY_PAID, or PAID based on the amount paid against the total.

```
====================================================================
                        BILLING & PAYMENTS
====================================================================
  [1] Generate Bill for Patient
  [2] View All Bills
  [3] View Pending Bills
  [4] View Bills by Patient ID
  [5] View Bill Details
  [6] Process Payment
  [0] Back to Main Menu
--------------------------------------------------------------------
Enter choice: 2

--- ALL BILLS ---

+--------+---------------------+------------+------------+------------+------------+-----------------+
| BillID | Patient             | Date       | Total(?)   | Paid(?)    | Balance(?) | Status          |
+--------+---------------------+------------+------------+------------+------------+-----------------+
| 9001   | Ramesh Kumar        | 2026-03-31 |    7800.00 |    5000.00 |    2800.00 | PARTIALLY_PAID  |
| 9002   | Arjun Patel         | 2026-03-31 |   35800.00 |       0.00 |   35800.00 | PENDING         |
| 9003   | Vikram Singh        | 2026-03-31 |    6300.00 |    6300.00 |       0.00 | PAID            |
+--------+---------------------+------------+------------+------------+------------+-----------------+

Press ENTER to continue...
```

```
 ==========================================================================
                              BILLING & PAYMENTS
 ==========================================================================
   [1] Generate Bill for Patient
   [2] View All Bills
   [3] View Pending Bills
   [4] View Bills by Patient ID
   [5] View Bill Details
   [6] Process Payment
   [0] Back to Main Menu
 --------------------------------------------------------------------------
 Enter choice: 5
 Enter Bill ID: 9003

 --- BILL DETAILS  [ID: 9003] ---
   Patient                    : Vikram Singh
   Date                       : 2026-03-31
   ---------------------------------------------
   DESCRIPTION                      AMOUNT (?)
   ---------------------------------------------
   Consultation Fee           :      500.00
   Bed Charges (2 days)       :     3000.00
   Medicines                  :     2000.00
   Laboratory Tests           :      800.00
   =============================================
   TOTAL                      :     6300.00
   PAID                       :     6300.00
   BALANCE                    :        0.00
   =============================================
   STATUS                     : PAID

 Press ENTER to continue...
```

5.5 Dashboard

- Total patients registered, currently admitted, and discharged
- Total doctors in the system

- Total appointments and number of upcoming (scheduled) appointments
- Total revenue collected and total pending balance across all bills

```
Enter choice: 5
==========================================================================
                    HOSPITAL DASHBOARD & REPORTS
==========================================================================

   ┌──────────────────────────────────────────────────────────────────┐
   │                        PATIENT STATISTICS                        │
   └──────────────────────────────────────────────────────────────────┘

  Total Patients                  : 7
  Currently Admitted              : 5
  Discharged                      : 2
  Total Doctors                   : 6
  Total Appointments              : 5
  Upcoming Appointments           : 5


   ┌──────────────────────────────────────────────────────────────────┐
   │                        FINANCIAL SUMMARY                         │
   └──────────────────────────────────────────────────────────────────┘

  Total Revenue Collected         : ? 11,300.00
  Total Pending Balance           : ? 38,600.00
  Pending Bills                   : 2
--------------------------------------------------------------------------

Press ENTER to continue...
```

6. Sample Data

The DataSeeder class pre-loads the system with realistic sample data when the application starts. This allows the system to be demonstrated immediately without entering records manually.

6.1 Doctors (Pre-Loaded)

ID	Name	Specialization	Experience	Patients Assigned
201	Dr. Anil Sharma	Cardiology	15 years	2
202	Dr. Priya Verma	Neurology	10 years	1
203	Dr. Rahul Gupta	Orthopedics	12 years	1
204	Dr. Sunita Patel	Pediatrics	8 years	1
205	Dr. Manoj Singh	General Surgery	20 years	0

6.2 Patients (Pre-Loaded)

ID	Name	Blood Group	Age	Disease
1001	Ramesh Kumar	A+	45	Hypertension
1002	Sita Devi	B+	32	Migraine
1003	Arjun Patel	O+	28	Fracture – Left Leg
1004	Meena Joshi	AB+	7	Fever & Cold

1005	Vikram Singh	A-	60	Chest Pain

Future Scope

Possible Future Enhancements

- Add JDBC support to save all data to a MySQL or SQLite database
- Build a Swing or JavaFX GUI for a desktop application
- Expose the service layer as a REST API using Spring Boot
- Add a login system with different roles: Admin, Doctor, and Receptionist
- Generate PDF discharge summaries and payment receipts automatically
- Send appointment reminders via email or SMS

8. Conclusion

This project demonstrates how Object-Oriented Programming principles can be used to design and build a real-world application in Java. The Hospital Management System covers the complete patient lifecycle from admission to discharge and billing, using five well-defined modules that each have their own service class and clearly defined responsibilities.

The use of encapsulation keeps all data safely inside model classes. The service layer provides clean abstraction so that the UI never has to deal with internal data structures. Java Streams make filtering and searching concise and readable. Enums provide a safe, clean way to track the state of appointments and bills without using magic strings or numbers.

The layered architecture of this project means it can be extended naturally. Adding a database layer, a graphical interface, or a web API would not require rewriting the existing service or model classes. This is the practical value of good OOP design – it produces code that is not just correct today but adaptable in the future.

9. References

1. Bloch, J. (2018). Effective Java, 3rd Edition. Addison-Wesley Professional.
2. Eckel, B. (2006). Thinking in Java, 4th Edition. Prentice Hall.
3. Oracle Corporation. Java SE 11 API Documentation. https://docs.oracle.com/en/java/javase/11/
4. Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design Patterns. Addison-Wesley.
5. VITyarthi – Object Oriented Programming Course Material, 2024-25.